



THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

CREATE CHANGE

High Dimensional Data Indexing

INFS 4205/7205 Advanced Techniques for High Dimensional Data

Yadan Luo

School of EECS, The University of Queensland

Recap: MLLM

- (M)LLM are a phenomenal piece of technology for knowledge generation and reasoning. They are pretrained on large amounts of **publicly available data**.



Llava 1.0

Llava 1.5

Llava-NEXT (1.6)

 **Meta**
Chameleon



LLaVA-Onevision



Qwen2-VL

 **deepseek**
Janus-Pro-7B

Use Cases

(V)Question-Answering

TextGen: Storytelling

ImageGen: Poster design

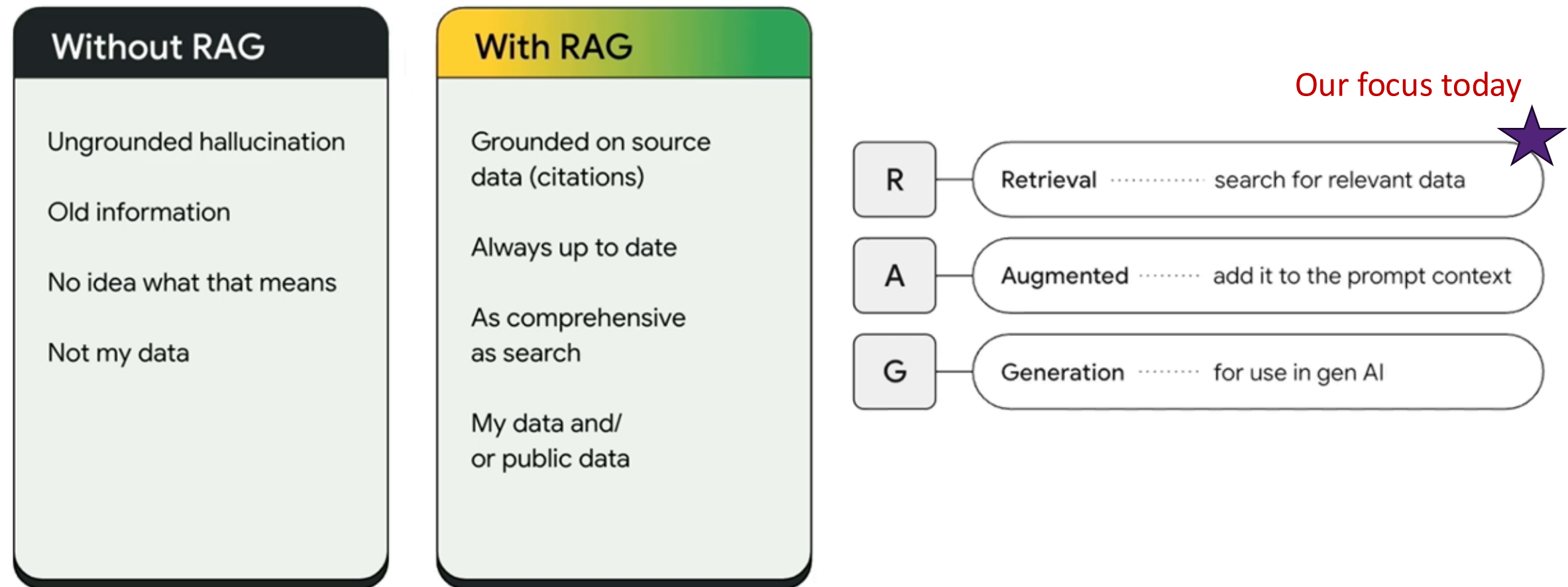
Planning

Summarization

...

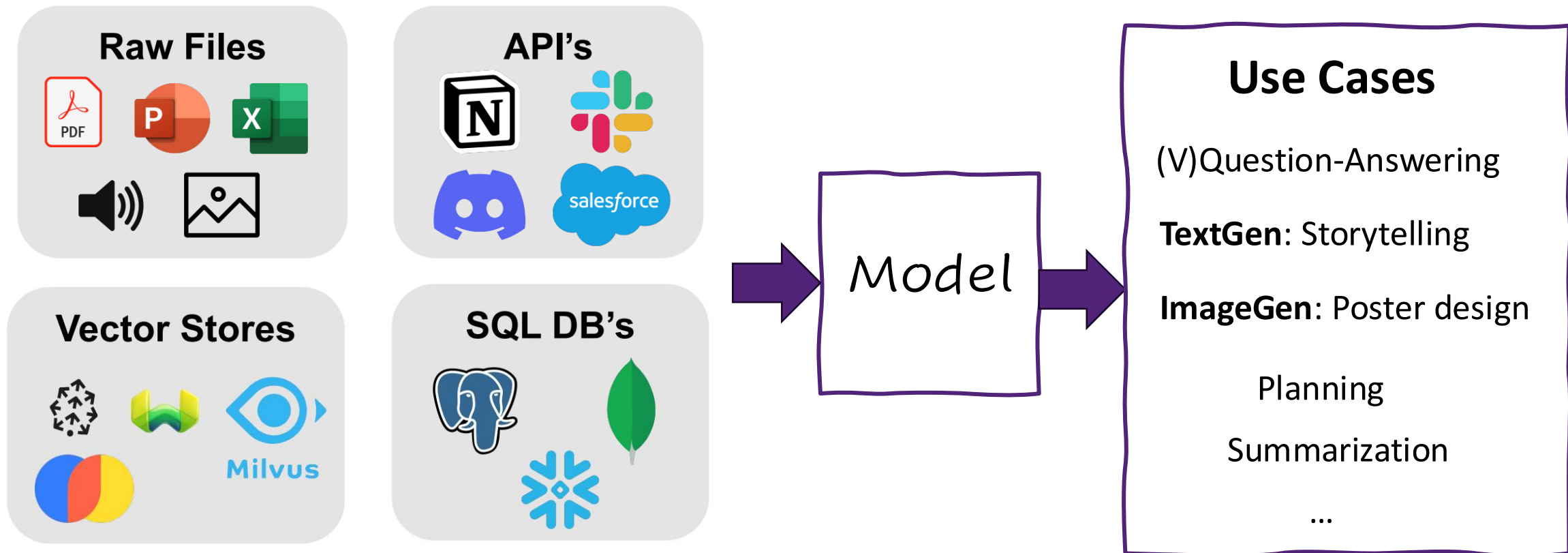
Reliable MLLM?

- Without RAG (week 8): (M)LLMs has to be **the only source of knowledge (pretraining)**

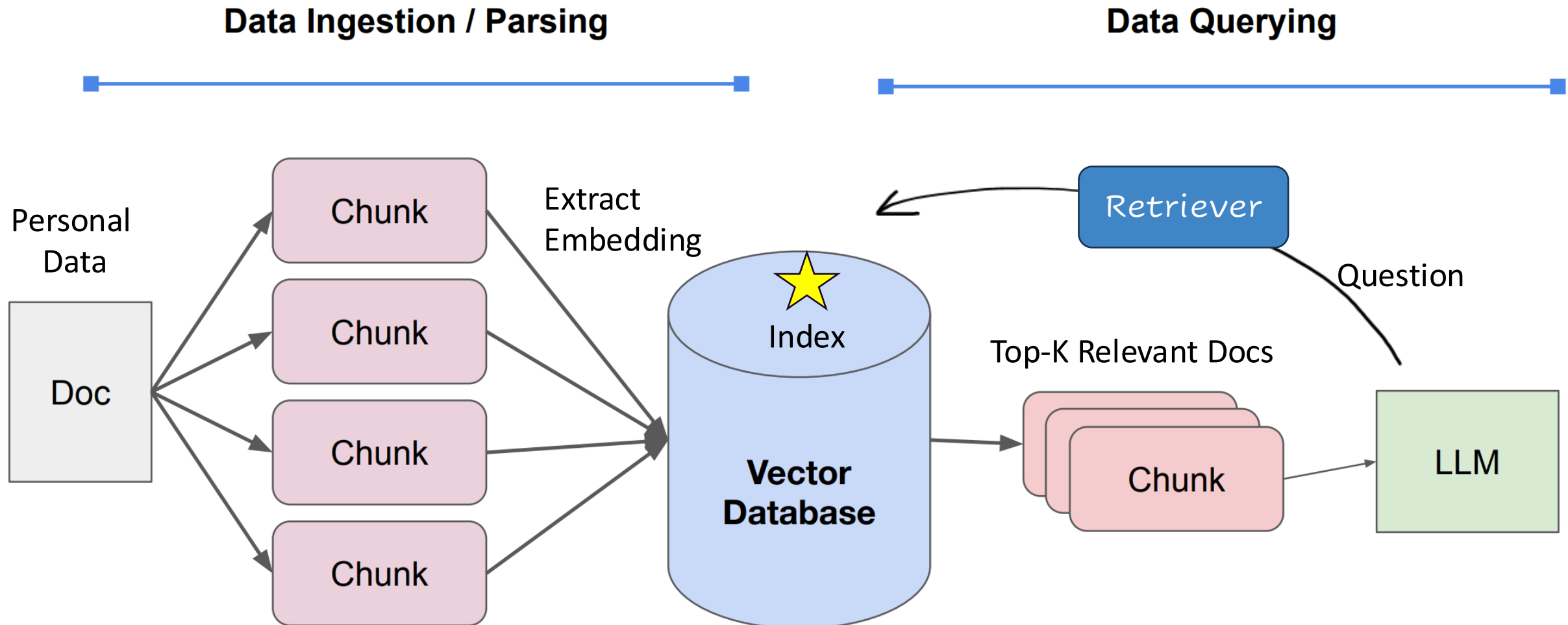


Reliable (M)LLM: From Generalist to Specialist

- How do we best augment (M)LLMs with our own **private data**?

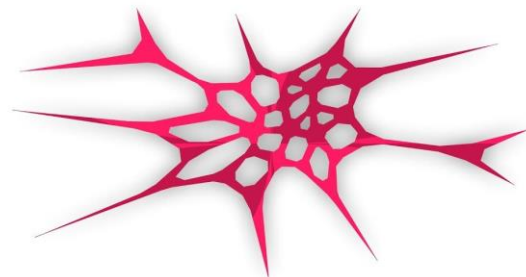
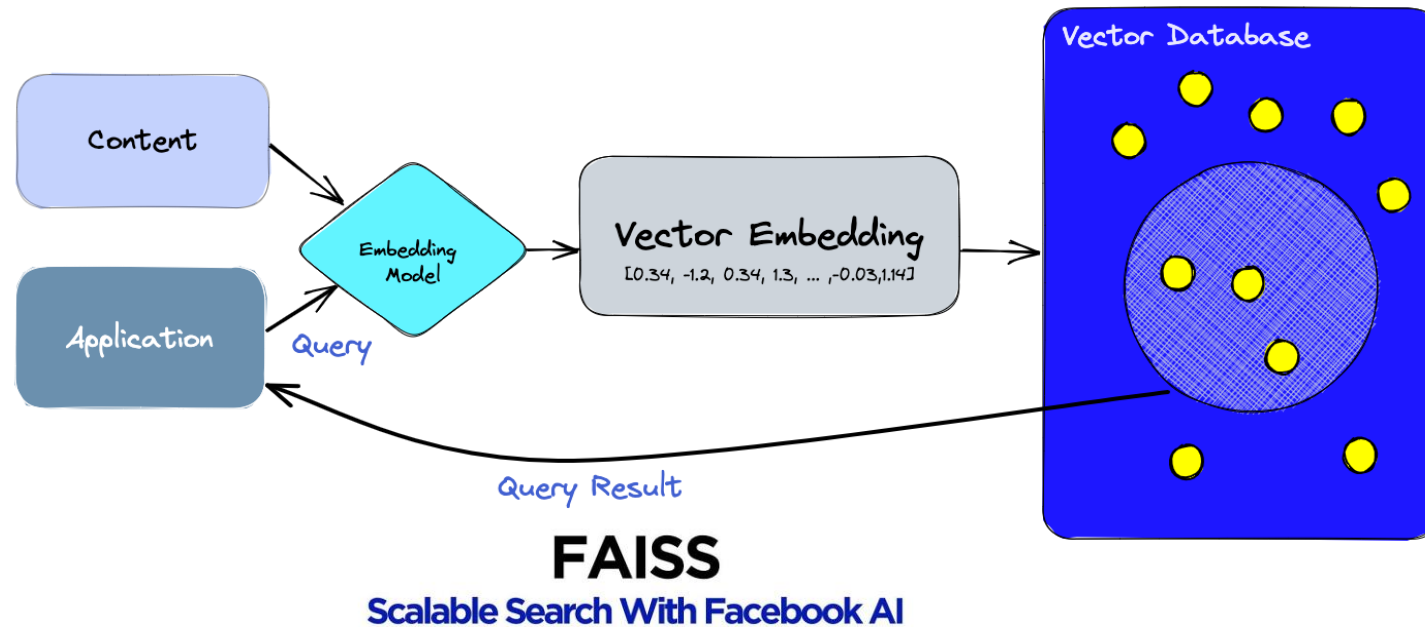


Multimodal RAG Pipeline (Week 8)

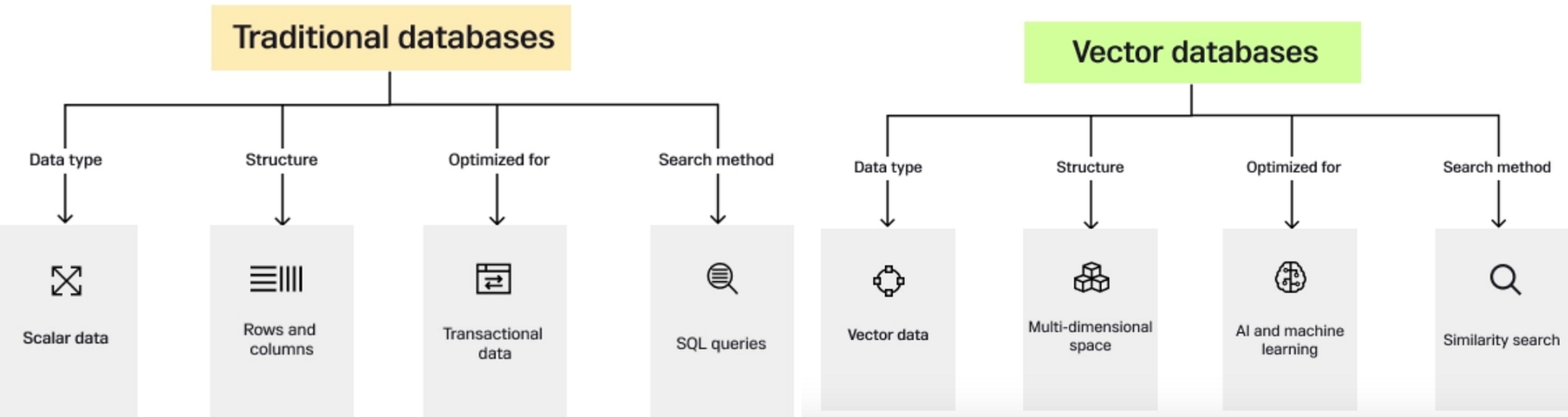


What is Vector DB?

- a **specialized database** designed to store, manage, and query high-dimensional vector embeddings



What is Vector DB?



- Traditional DB: **discrete**, **scalar** data types (numbers and strings) in rows/ columns.
- Vector DB: **high-dim** vector data (embeddings) for **similarity search**

Why Similarity Search?

Method	Works Well In	Performance Degrades In	Notes
KD-Tree	$\leq 10D$	$\geq 20D$	Poor recall, linear scan-like
R-Tree	2D–3D	$\geq 5D$	Overlapping bounding boxes

- If we stick to exact kNN search... (recall in Lec1, as dimension **goes up**, volume grows **exponentially**, traditional indexing (e.g., KD-trees collapses into $O(N)$ **linear scans**)



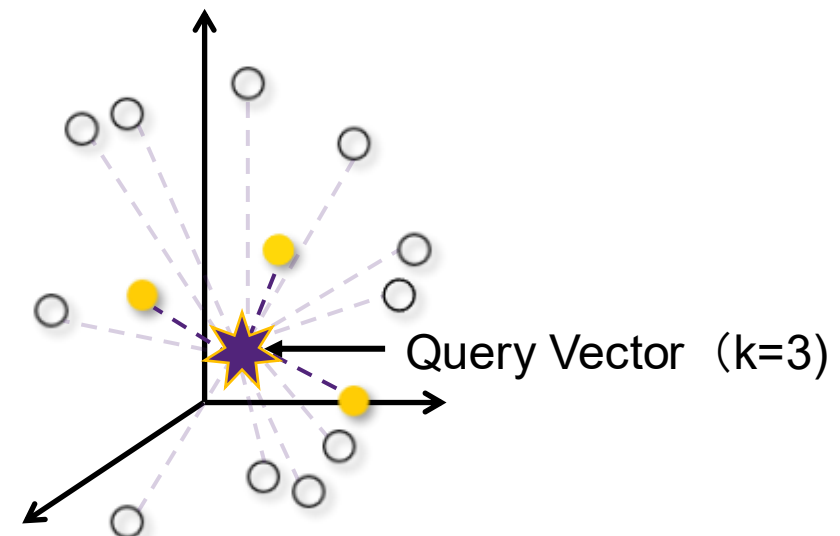
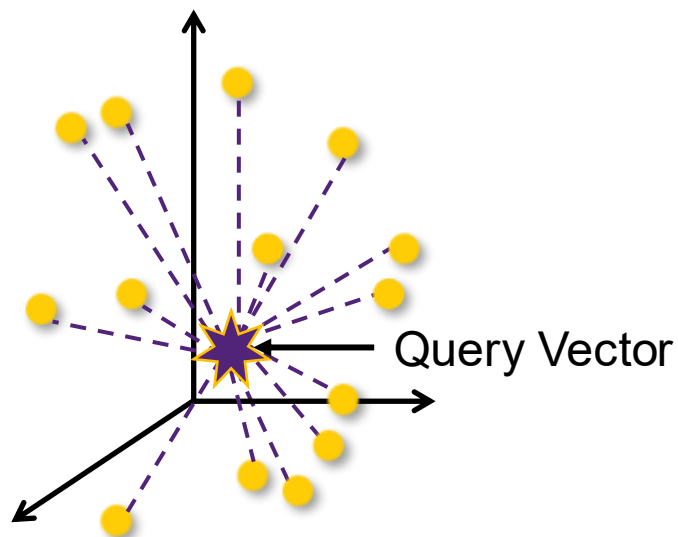
Exact k-Nearest Neighbors	The Math	The Verdict
Requires a full linear scan of the entire dataset.	50,000,000 records $\times$ 4096 dimensions = Computational paralysis.	Brute force is the enemy of real-time AI.

Why Similarity Search?

- *If we stick to exact kNN search...*

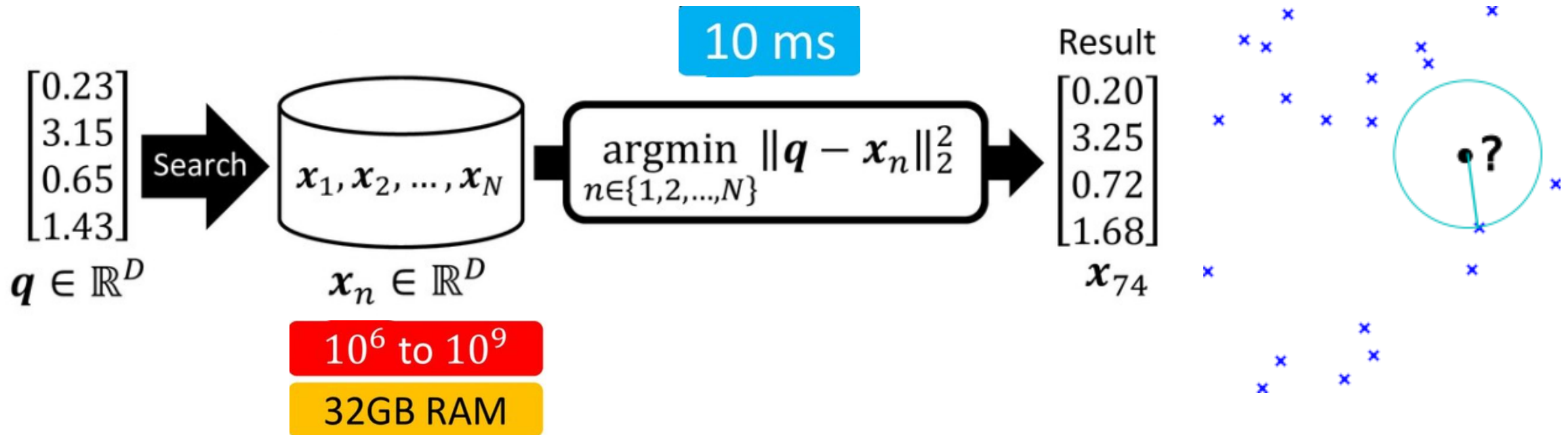
Flat Index (Brute Force)

- Flat indices are a direct representation of the vector embedding
- Deliver the best accuracy of all indexing methods, but notoriously slow
- Search is exhaustive: it's performed from the query vector across *every single vector embedding* and distances are calculated for each pair.



Approximate Nearest Neighbour Search (ANN)

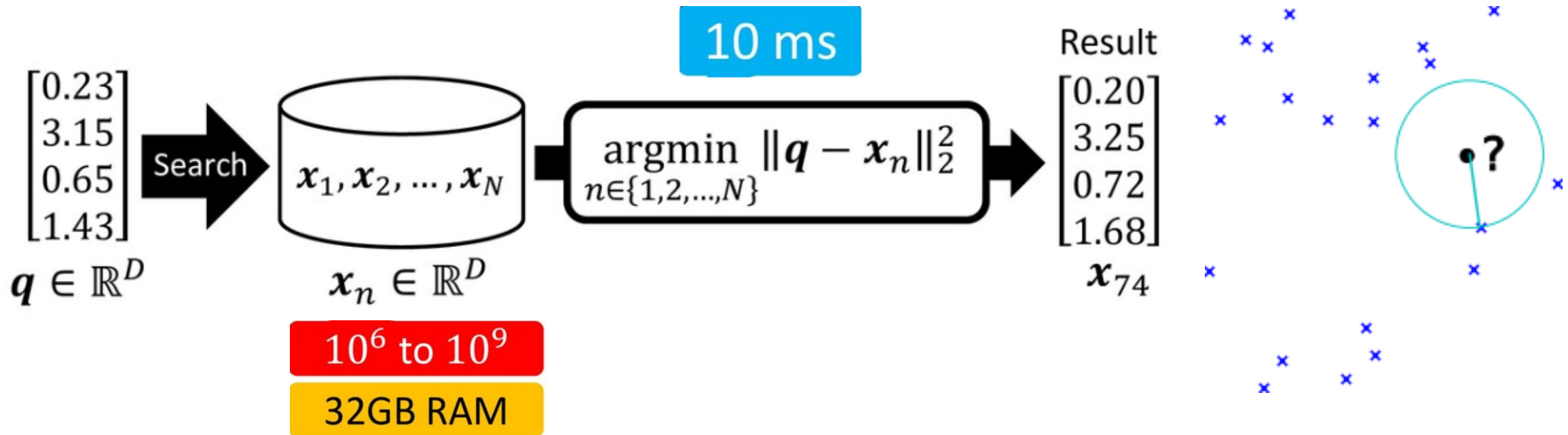
- Allowed to return points, whose distance from the query is at most c times the distance from the query to its nearest points



- Faster Search:** billion-scale data on memory
- Trade-offs:** runtime, accuracy, memory-consumption

Approximate Nearest Neighbour Search (ANN)

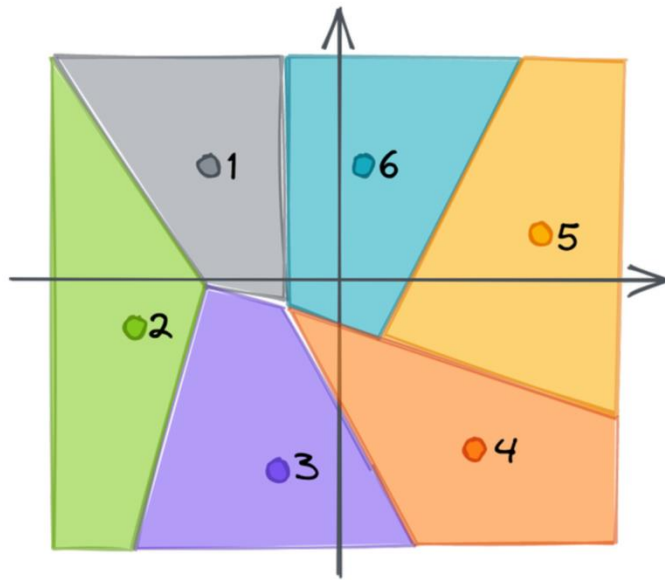
- Allowed to return points, whose distance from the query is at most c times the distance from the query to its nearest points



- Faster Search:** billion-scale data on memory
- Trade-offs:** runtime, accuracy, memory-consumption

How to achieve ANN?

Inverted files (IVF)



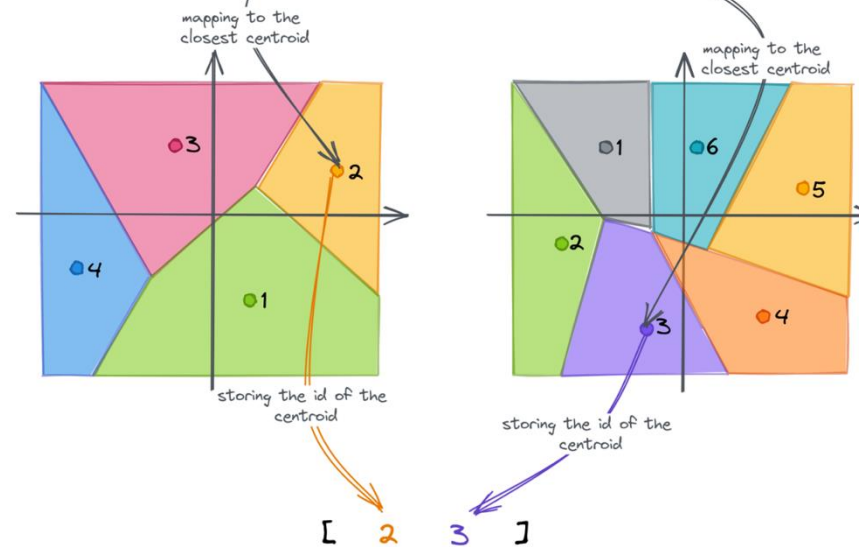
Partitioning

split the whole data into several clusters using K-means

$[-0.25 \ -0.45 \ 0.98 \ -0.62 \ -0.98 \ -0.76 \ 0.08 \ 0.57]$

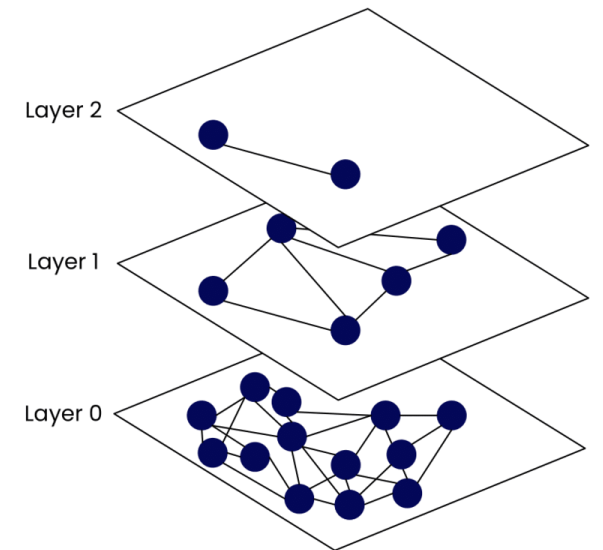
cutting into chunks

$[-0.25 \ -0.45 \ 0.98 \ -0.62] \ [-0.98 \ -0.76 \ 0.08 \ 0.57]$



Quantization

(e.g., Product Quantization)
compress the vector

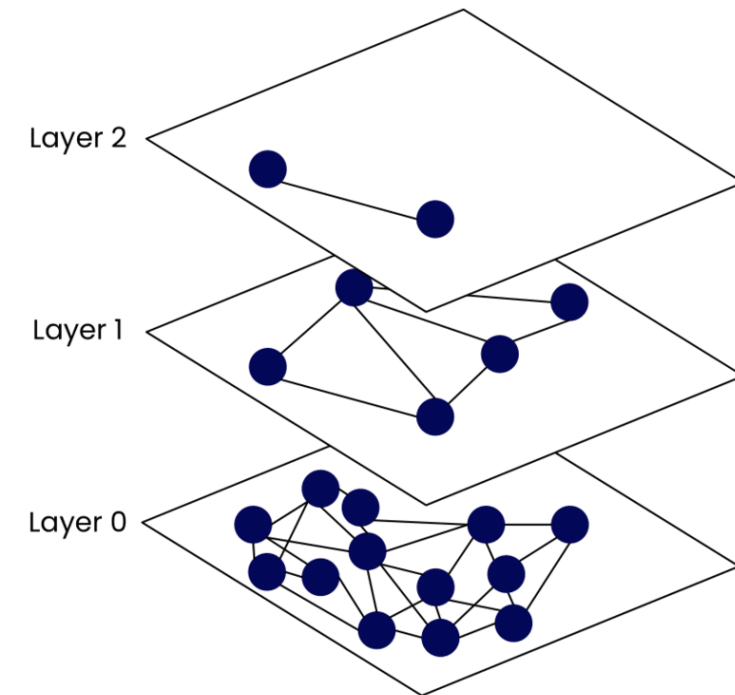


Graphs

(e.g., HNSW) navigate the space

Hierarchical Navigable Small Worlds (HNSW)

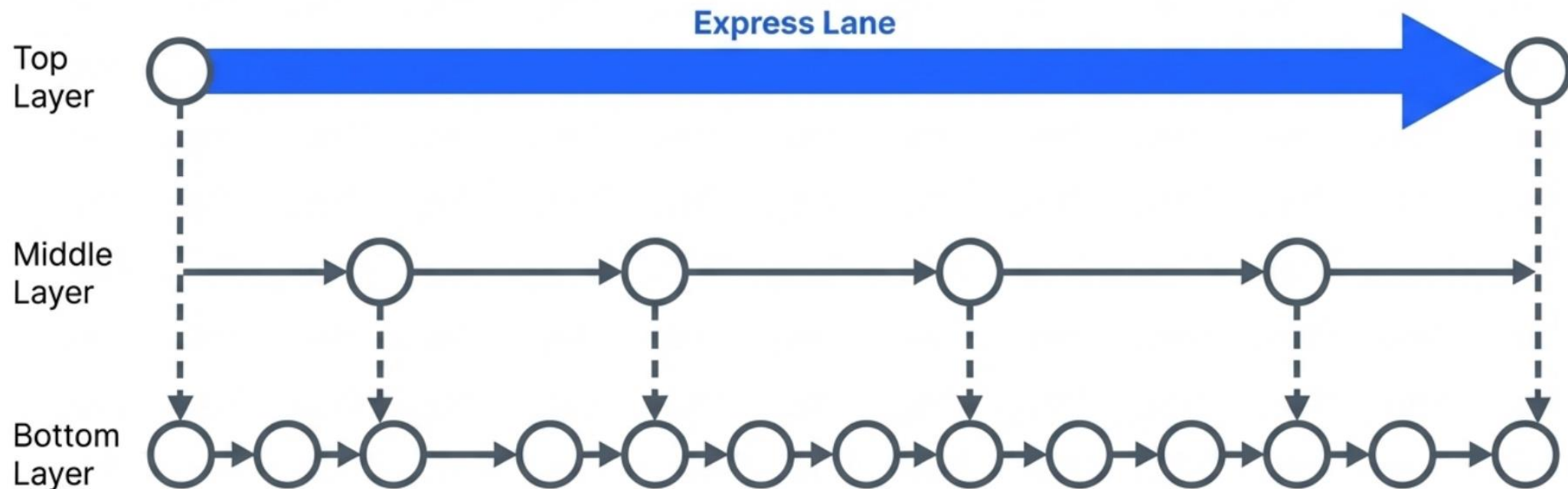
- “proximity” graph indexing; construct a network-like structure
 - **Nodes:** vector embeddings
 - **Edge:** relationship between embeddings (often Euclidean distance).
- Starting at a predefined “**entry point**”, the algorithm **traverses** connected friend vertices until it finds the nearest neighbour to the query vector



Hierarchical Navigable Small Worlds (HNSW)

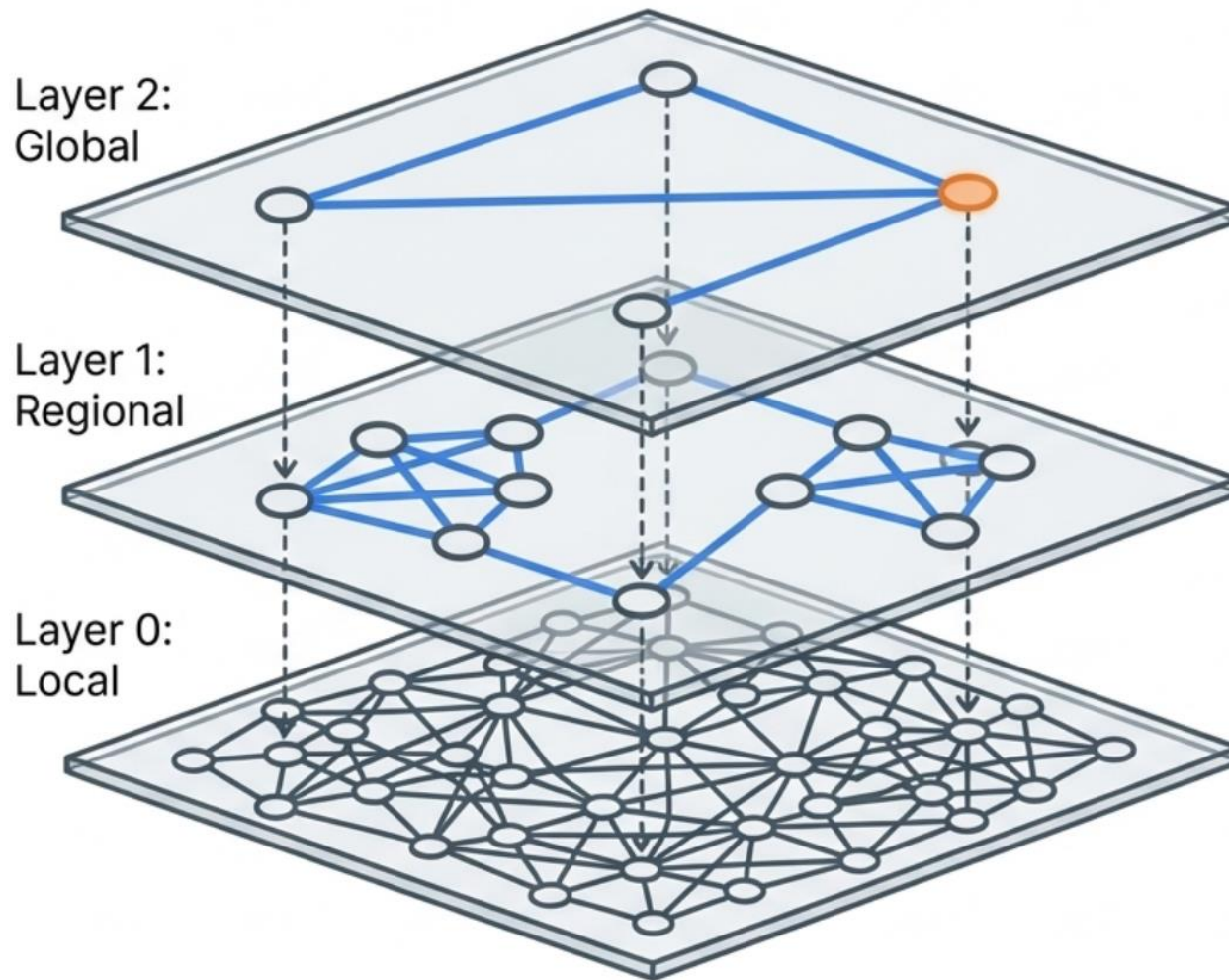
The Challenge: How to search a sorted list in $O(\log N)$ time instead of $O(N)$?

Motivated by 1D Skip List...



The Mechanism: Add hierarchical express lanes for massive long jumps.

Hierarchical Navigable Small Worlds (HNSW)



The Gold Standard

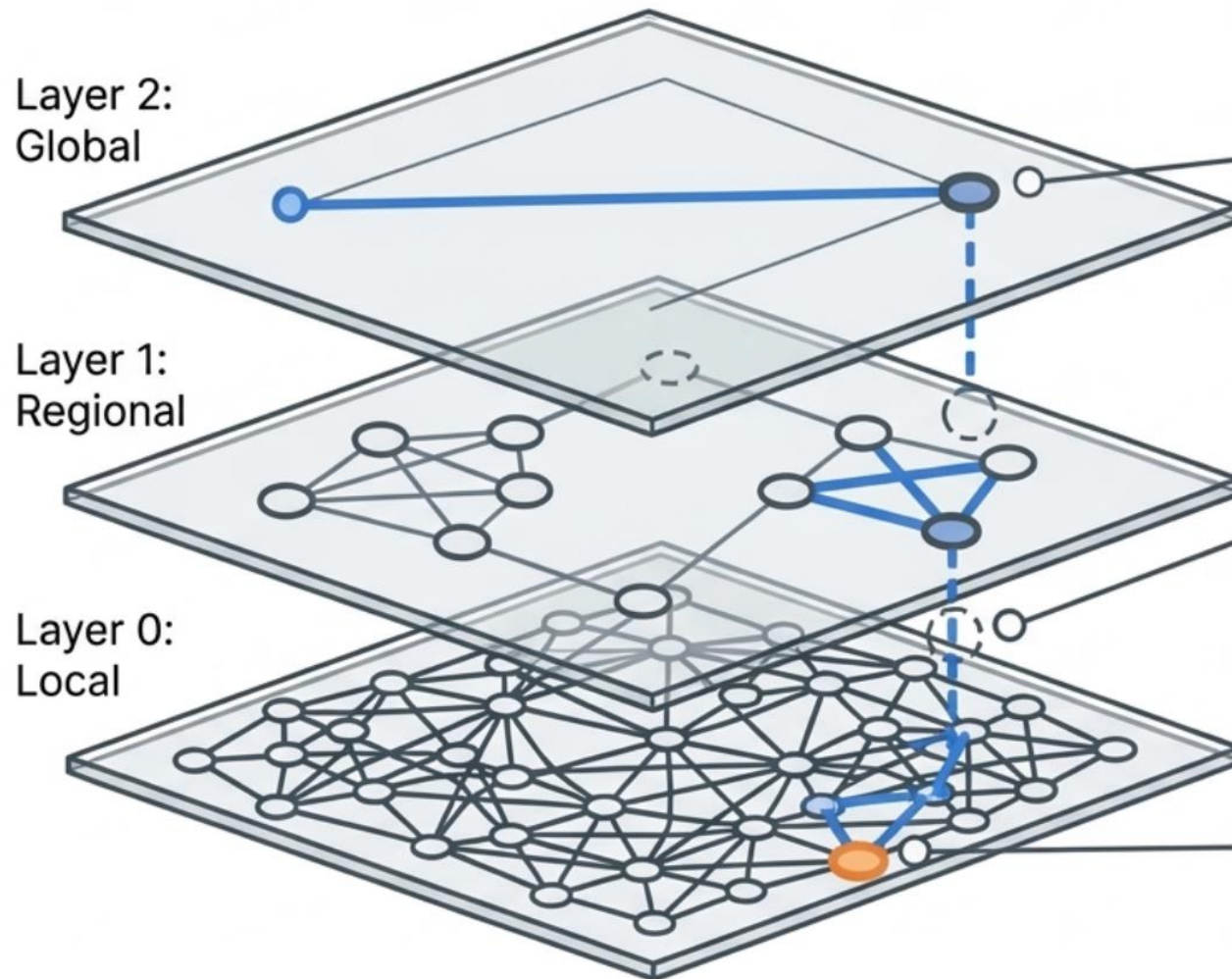
The dominant algorithm for modern Vector Search.

The Architecture

Extends the 1D skip list concept into high-dimensional geometric graphs.

Layer 2	Global navigation (Long jumps)
Layer 1	Regional navigation
Layer 0	Local, exact neighborhood search

Hierarchical Navigable Small Worlds (HNSW)



The Logic

Execute a greedy search at each individual layer.

The Drop

Descend to the next, denser layer only when a local minimum is reached.

The Result

Lightning-fast, $O(\log N)$ traversal.

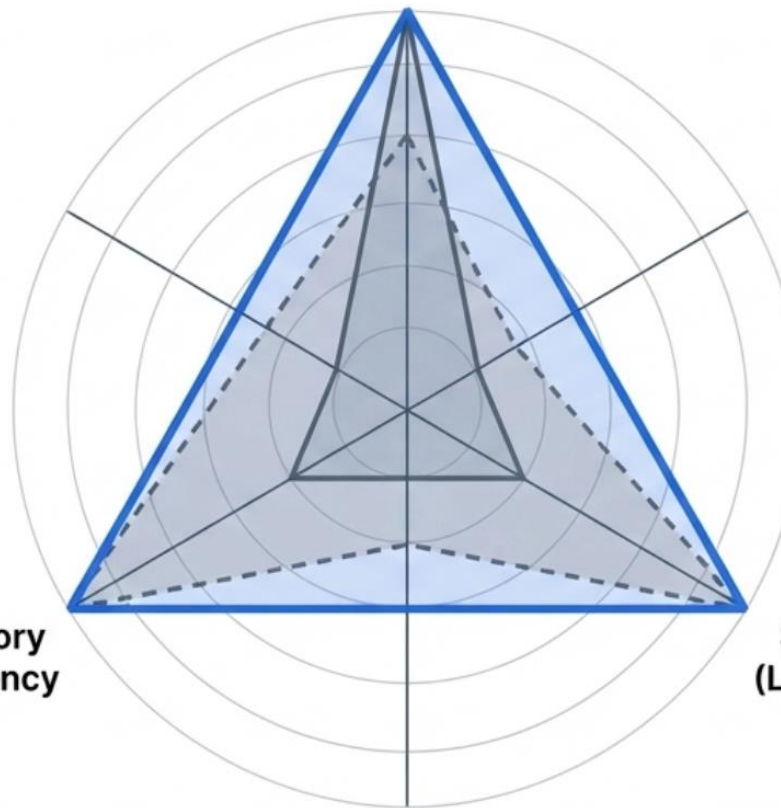
Hierarchical Navigable Small Worlds (HNSW)

Algorithm Summary: The Trade-off Matrix

Flat (Exact)

Perfect recall, slow speed, high memory overhead.

Recall (Accuracy)



-  Pinecone Proprietary composite index
-  milvus /  zilliz Flat, Annoy, IVF, HNSW/RHNSW (Flat/PQ), DiskANN
-  Weaviate Customized HNSW, HNSW (PQ), DiskANN (in progress....)
-  drant Customized HNSW
-  chroma HNSW
-  LanceDB IVF (PQ), DiskANN (in progress....)
-  vespa HNSW + BM25 hybrid
-  Vald NGT
-  elasticsearch Flat (brute force), HNSW
-  redis Flat (brute force), HNSW
-  pgvector IVF (Flat), IVF (PQ) in progress...

IVF-PQ

Good recall, high speed, ultra-low memory.

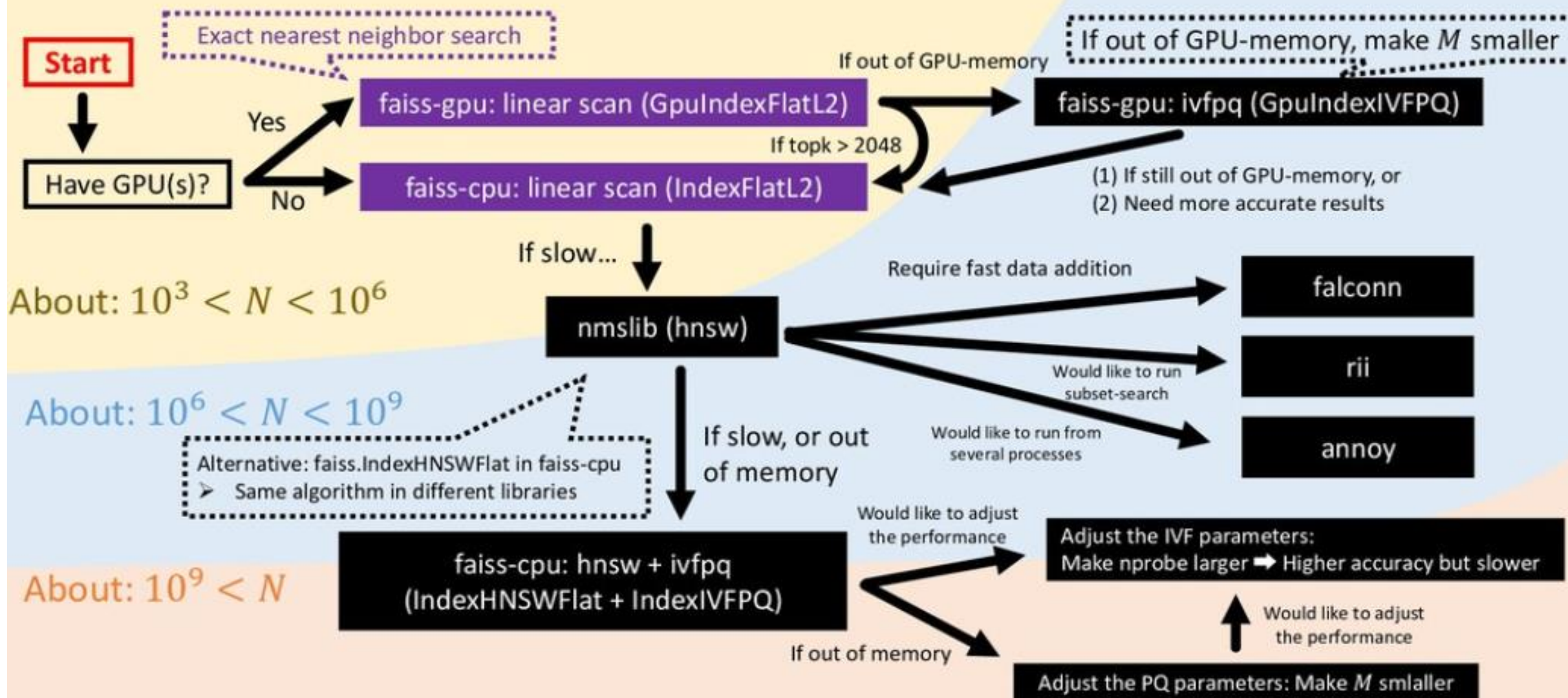
Memory
Efficiency

Speed
(Latency)

HNSW

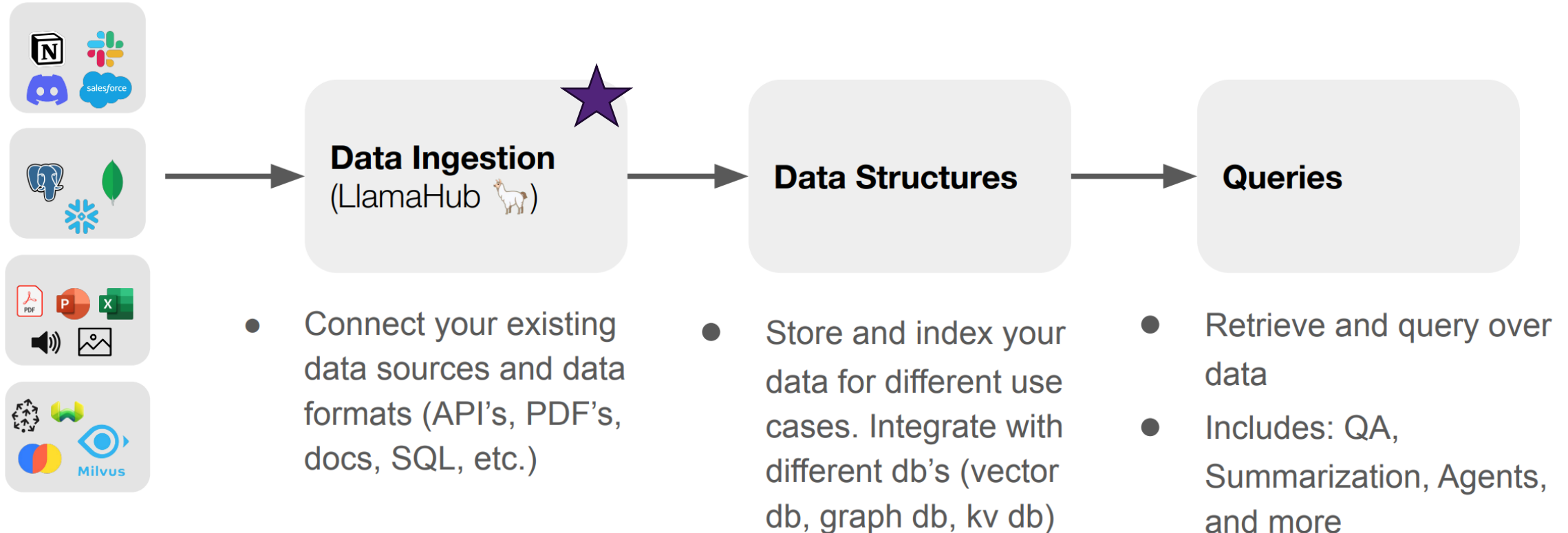
Extremely high recall, ultra-high speed, massive memory requirement.

cheat-sheet for ANN in Python (as of 2020. Can be installed by conda or pip)



LLamaIndex: data framework for (M)LLM

- Data Management and Query Engine for your (M)LLM application
- Offers components across the data lifecycle: ingest, index, and query over data



Data Connectors: powered by LlamaHub

- Easily ingest any kind of data, from anywhere
- Growing support for multimodal documents (e.g. with inline images)

```
from llama_index import download_loader
import os

NotionPageReader = download_loader('NotionPageReader')

integration_token = os.getenv("NOTION_INTEGRATION_TOKEN")
page_ids = ["<page_id>"]
reader = NotionPageReader(integration_token=integration_token)
documents = reader.load_data(page_ids=page_ids)
```

**<10 lines of code to
ingest from Notion**



Demo

PDF RAG — Index & Embedding Benchmark

Compare FAISS indexing strategies and embedding models on your PDF.

PAGES
116

CHUNKS
175

FIGURES
151

AVG CHUNK LEN
1006

Document Index Comparison Embedding Comparison

Parsed Document

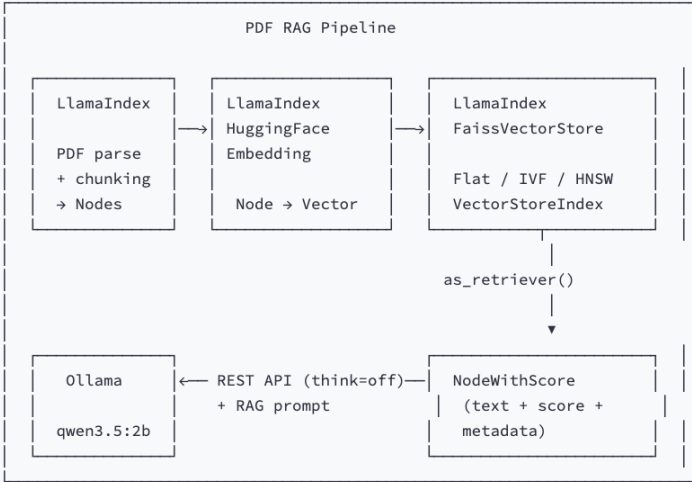
Text Chunks

▼ Chunk 1 · page 1 · 364 chars

107 recipes
all with photos
Chocolate Lover's
Dream Cake, page 83
Celebrating Be/tt y's
classic dishes and
new favorites!
all-time
/ninelineing/zerolineing
YEARS/spacelineingOF
BETTY/spacelineing
CROCKER!
Display until October
/onelineing/fourlineing/commalineing/spacelineing/twolining/zerolineing/onelineing/onelineing
V volume /twolineing/onelineing, Number /fivelineingA

> Chunk 2 · page 2 · 0 chars

Extracted Figures



Library	Role in this demo
LlamaIndex — SimpleDirectoryReader + SentenceSplitter	PDF → Nodes (text chunks with metadata)
LlamaIndex — HuggingFaceEmbedding	Embed nodes (wraps sentence-transformers)
LlamaIndex — FaissVectorStore	Wraps a raw FAISS index for add / query / persist
LlamaIndex — VectorStoreIndex + StorageContext	Connects vector store + docstore; provides as_retriever()
FAISS (faiss-cpu)	Underlying index engine (Flat / IVF / HNSW) — accessed through FaissVectorStore
Ollama REST API (think=false)	Assemble RAG prompt, call local LLM — thinking disabled for speed

Why FaissVectorStore instead of raw FAISS? — It integrates FAISS into LlamaIndex's storage/retrieval pipeline: StorageContext manages the vector store + docstore, and vectorStoreIndex.as_retriever() handles query embedding and result mapping automatically. You still control the raw FAISS index type and params.

Demo

Document

Index Comparison

Embedding Comparison

How It Works

FAISS Index Comparison

Flat (exact) vs IVF (inverted file) vs HNSW (graph-based) — retrieval **and** RAG answer impact.

Embedding model

all-MiniLM-L6-v2

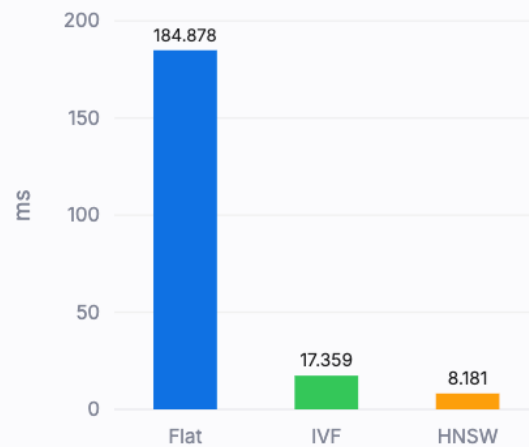
Search query

how to cook devil eggs?

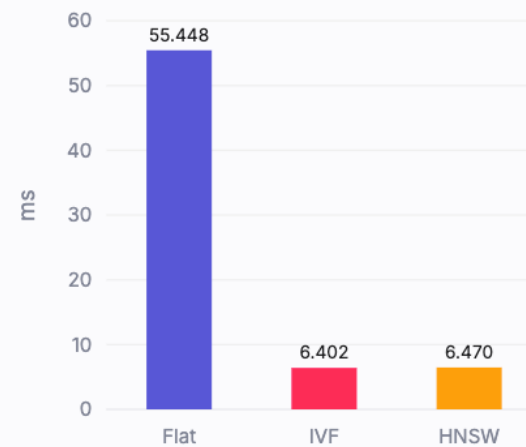
Embedded 175 chunks (384-d) in 1.13 s with all-MiniLM-L6-v2

Performance

Build Time



Query Time



Recall@5



Demo

LLM Response Time



Flat (Exact)

27.0s · 6507 ctx chars · nodes ['86543628', '2384620d', '4ceaecb3', '4ed63bc3', '76ee2475']

Based on the provided context, there are two distinct recipes for "Deviled Eggs" found in the document:

1. Shrimp Deviled Eggs

- **Prep Time:** 30 Minutes
- **Cooking Time:** 30 Minutes
- **Ingredients:**
 - 6 hard-cooked eggs
 - 2 medium green onions, thinly sliced
 - ¼ cup reduced-fat mayonnaise or salad dressing
 - 1 tablespoon white vinegar
 - ¼ teaspoon salt
 - ¼ teaspoon red pepper sauce
 - ½ cup coarsely chopped cooked salad shrimp (thawed if frozen)

IVF (Approx)

8.2s · 6630 ctx chars · nodes ['2384620d', '4ceaecb3', '4ed63bc3', '76ee2475', 'd5ecd2db']

Based on the provided context, there is no recipe for "devil eggs."

The document contains two recipes that use the term "devil's food" (a cake mix), but neither is for eggs:

1. **Gluten Free Zucchini-Devil's Food Snack Cake** (Passage 2): This recipe uses a cake mix, zucchini, and eggs to make a cake, not deviled eggs.
2. **Pull-Apart Turtle Cupcakes** (Passage 3): This recipe uses a cake mix, frosting, and candies to make cupcakes shaped like turtles, not deviled eggs.

The only recipe that actually makes "devil eggs" is the **Deviled Eggs** recipe in **Passage 1**, which is not included in the provided text.

HNSW (Graph)

Same chunks as Flat (Exact) — skipped LLM call

Based on the provided context, there are two distinct recipes for "Deviled Eggs" found in the document:

1. Shrimp Deviled Eggs

- **Prep Time:** 30 Minutes
- **Cooking Time:** 30 Minutes
- **Ingredients:**
 - 6 hard-cooked eggs
 - 2 medium green onions, thinly sliced
 - ¼ cup reduced-fat mayonnaise or salad dressing
 - 1 tablespoon white vinegar
 - ¼ teaspoon salt
 - ¼ teaspoon red pepper sauce
 - ½ cup coarsely chopped cooked salad shrimp (thawed if frozen)
 - 1 tablespoon cocktail sauce

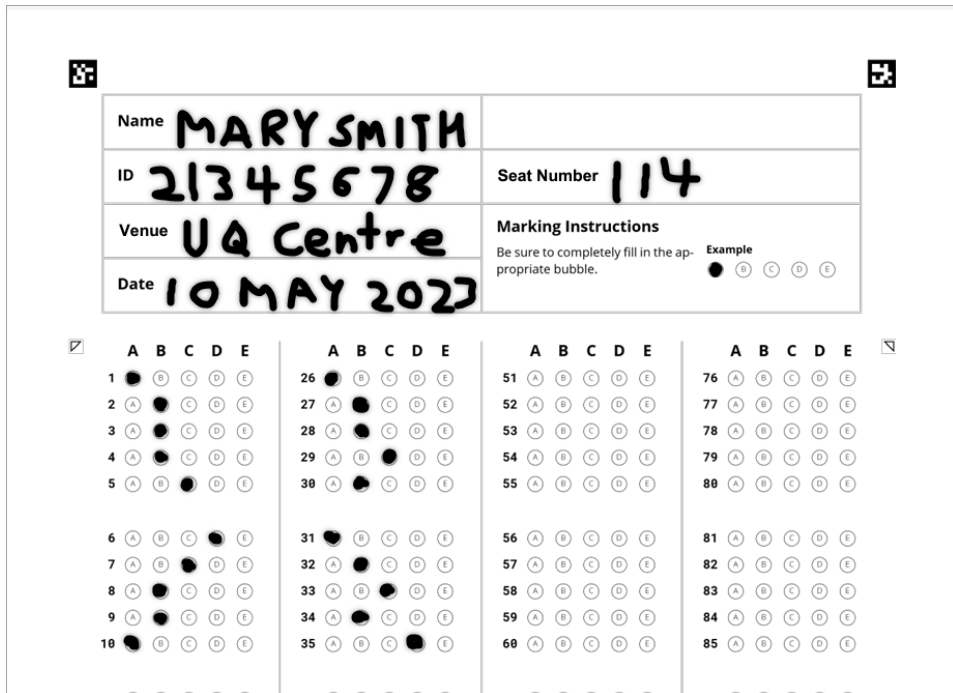
What we covered today...

After today's class, you should be able to

- understand the **RAG** framework using private, up-to-date data to prevent hallucinations.
- distinguish between traditional scalar databases (SQL) and multi-dimensional Vector Databases.
- why Approximate Nearest Neighbour (ANN) search is essential for industrial scale.
- **Core Indexing:** Space clustering (**IVF**) and Graph-based routing (**HNSW**).
- **Data Pipelines:** Ingesting, chunking, and indexing documents using **LlamaIndex**.

Next Week: Happy Semester Break 😊

Gradescope Bubble Sheet tips



The image shows a Gradescope Bubble Sheet with the following fields filled in:

Name	MARY SMITH	
ID	21345678	Seat Number 114
Venue	UQ Centre	
Date	10 MAY 2023	

Marking Instructions: Be sure to completely fill in the appropriate bubble. Example: ● ○ ○ ○ ○

The bubble sheet contains 85 numbered rows (1-85) and five columns of bubbles labeled A, B, C, D, E. Some bubbles are pre-filled with a black dot, indicating the correct answer for each row.

1. Your exam will use a Gradescope Bubble Sheet
2. Enter your **FIRST NAME & LAST NAME** in block capitals
3. Enter your eight digit UQ student ID
4. For central exams enter the venue and your seat number
5. Enter the date of your exam
6. Fill in the bubbles using a **2B pencil**
7. To change an answer use an **eraser**



THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

CREATE CHANGE

THANKS!

Do you have any questions?

uqyluo@uq.edu.au

The University of Queensland

